252-0027

Introduction to Programming

Exercises

Week 6: References, Recursion, Precondition

Jan Leonard
Department of Computer Science
ETH Zurich

Organization

- My name: **Jan Leonard**
- For questions: **leonardj@ethz.ch**
  - Please put «**[EProg26]**» in the subject
- New exercises: Tuesday evening (normally)
- Submit exercises by Tuesday evening (23:59) following week
  - Always submit via Git
  - Solutions in a separate Git project



Eprog TA — HS 2026

janleonard115.github.io

Plan for today

- Preview
- Feedback
- References
- Recursion
- Logical Reasoning
- Weakest Precondition
- Kahoot

Preview

`ArrayUtil.zeroInsert(int[] x)`

Exercise 1: Prefix Construction

Given are two strings `s` and `t` and an integer `n` with $n \geq 0$. Write a method that returns whether `s` is a concatenation of at most `n` prefixes of `t`.

Examples:

- `s = "abcababc"`, `t = "abc"`, `n = 4`: The method should return `true`, because `"abc"` and `"ab"` are prefixes of `t` and `s` is a concatenation of `"abc"`, `"ab"`, `"abc"`.
- `s = "abcbcabc"`, `t = "abc"`, `n = 4`: The method should return `false`, because `"bc"` is not a prefix of `t`.
- `s = "abab"`, `t = "abac"`, `n = 2`: The method should return `true`, because `"ab"` is a prefix of `t` and `s` is a concatenation of `"ab"`, `"ab"`.

Implement the method `isPrefixConstruction(String s, String t, int n)` in the class `PrefixConstruction`. The method has three arguments: the two strings `s` and `t` and the integer `n`. You may assume that n is greater than or equal to 0. You will find tests in the file "PrefixConstructionTest.java".

Tip: Solve the task recursively.

Tip: Check s = "ababa" t = "aba" n = 2 on paper.

Exercise 5.2: Arrays

If Exercise 6.1 is too difficult: Solve this exercise first!

1. Implement the method `ArrayUtil.zeroInsert(int[] x)` in the file "ArrayUtil.java". The method takes an array `x` as an argument and returns an array. The returned array should have the same values as `x`, except: if a positive number directly follows a negative number or if a negative number directly follows a positive number, then a 0 is inserted between them.

   Examples:

   - If `x` is equal to `[3, 4, 5]`, then `[3, 4, 5]` is returned.
   - If `x` is equal to `[3, 0, -5]`, then `[3, 0, -5]` is returned.
   - If `x` is equal to `[-3, 4, 6, 9, -8]`, then `[-3, 0, 4, 6, 9, 0, -8]` is returned.

   Try to implement the method recursively.

Exercise 5: Bonus Exercise

Attention: This task gives bonus points (see "Leistungskontrolle" at `www.vvz.ethz.ch`). The task must be solved independently and alone. Submission is done as usual by pushing to your Git repository on the ETH server. The last push before the submission deadline is binding. Even if you commit before the deadline but push after the deadline, this counts as a late submission. Please also read the general rules.

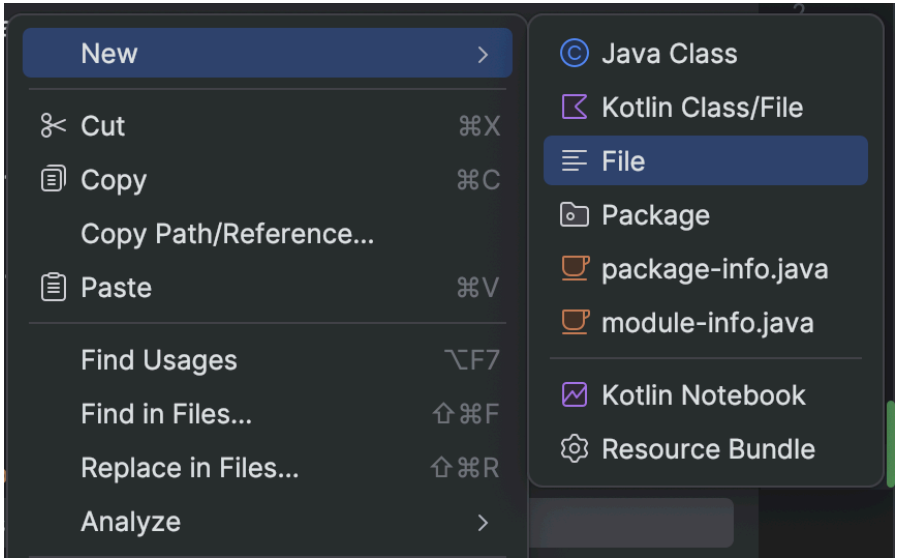
Feedback

Feedback

- Starting with u05, you tell the TAs where you would like feedback.
- For this, create a requestfeedback.txt file in the uXX folder, not in src / test / resources.
- In the file, write the exercises for which you want feedback. (Bonus is graded separately)
- If you want I will also give you feedback on the bonus task.
  - Just write that in the file

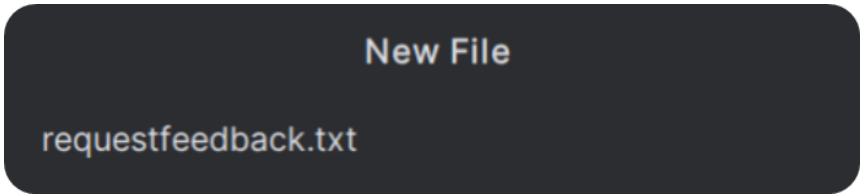
Feedback Datei erstellen

1. **Right-click project folder uXX.**
2. **New -> File.**



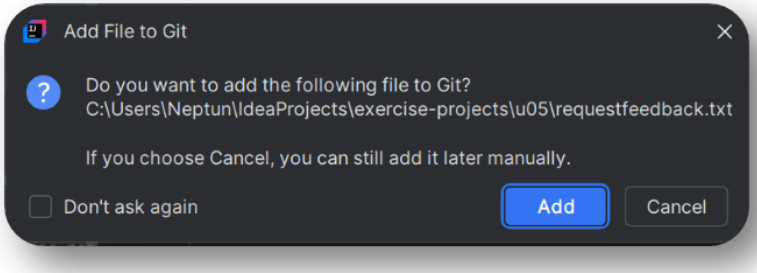
Feedback Datei erstellen

1. **Right-click project folder uXX.**
2. **New -> File.**
3. **Name the file requestfeedback.txt in the window that appears.**



Feedback Datei erstellen

1. Right-click project folder uXX.
2. New -> File.
3. Name the file requestfeedback.txt in the window that appears.
4. Choose Add in the appearing "Add File to Git" window



References

Quiz: References

```
int[] a = {1, 2};
int[] b = a;
a = null;
System.out.println(b[0]);
```

Value vs Reference

- Variables contain either a value or a reference.
- Variables never contain objects.
- References point to the object in memory.

Value vs Reference - Example

```
1  public class Example {
2      public static void setX(int x) {
3          x = 2;
4      }
5
6      public static void main(String[] args) {
7          int x = 3;
8          setX(x);
9          System.out.println(x); ⟶ 3
10     }
11 }
```

Value vs Reference - Example

```
1  public class Example {
2      public static void setX(int x) {
3          x = 2;
4      }
5
6      public static void main(String[] args) {
7          int x = 3;
8          setX(x);
9          System.out.println(x);
10     }
11 }
```

Scope setX

Scope main

Value vs Reference

- References also follow value semantics:
  - In Java, variables that refer to objects are actually treated as references.
  - When one reference is assigned to another, only the memory address (reference) is copied, not the actual object.
  - Each reference stores the address of the object it points to.
- Only the objects themselves enable reference semantics:
  - Several references can point to the same object in memory (e.g. A obj1 = new A(); A obj2 = obj1;).
  - References in Java are not independent copies of the object, but pointers to the same memory location.

Value vs Reference - Example

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```

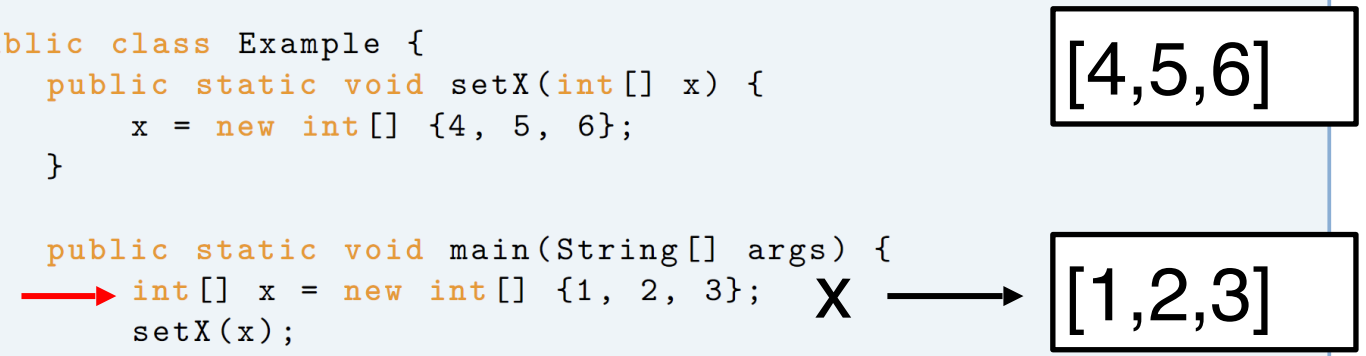
[4,5,6]

X

[1,2,3]

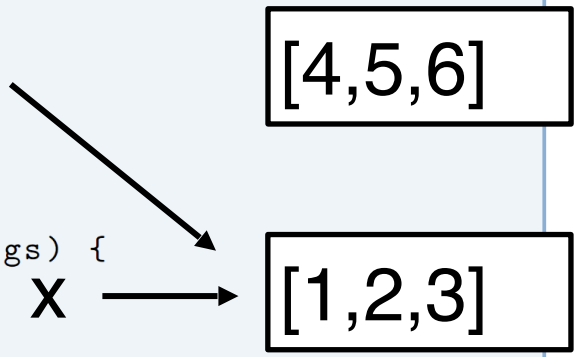
Value vs Reference - Example

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```



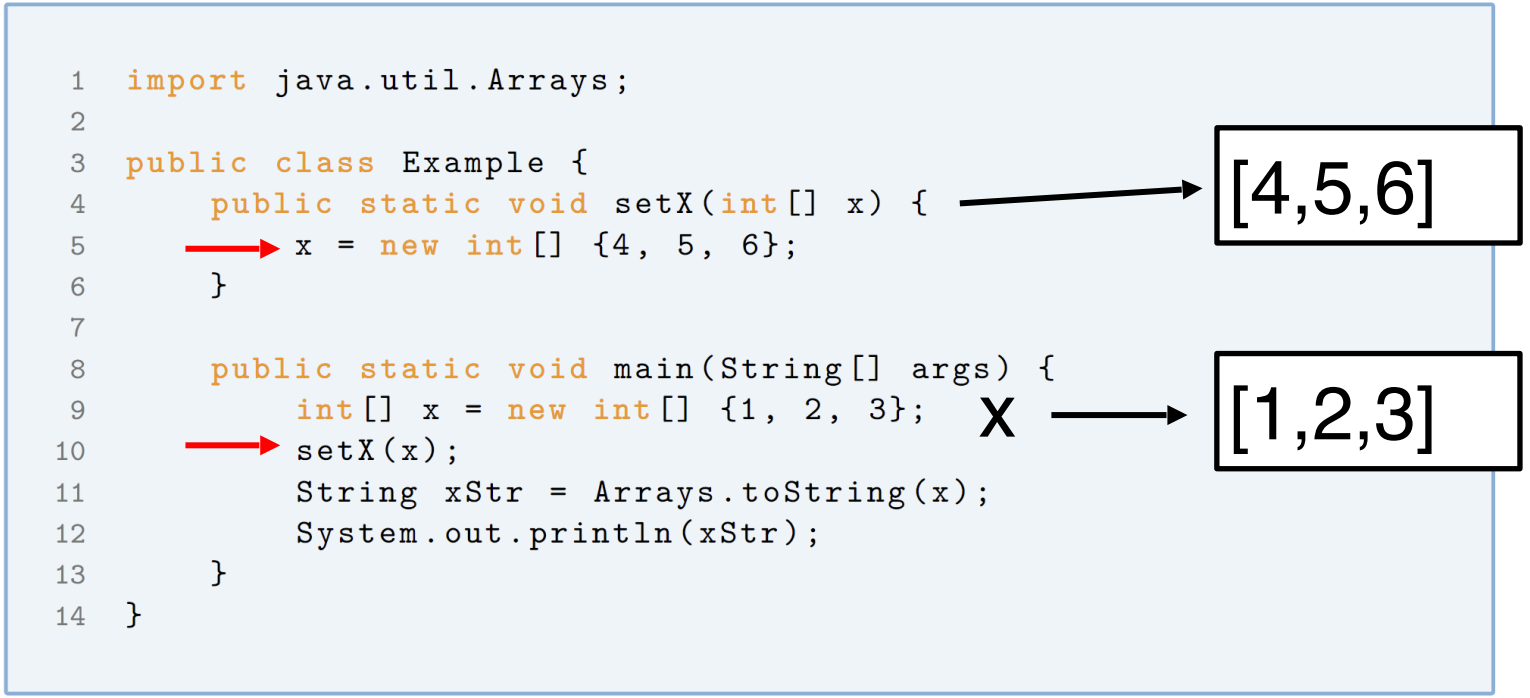
Value vs Reference - Example

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```



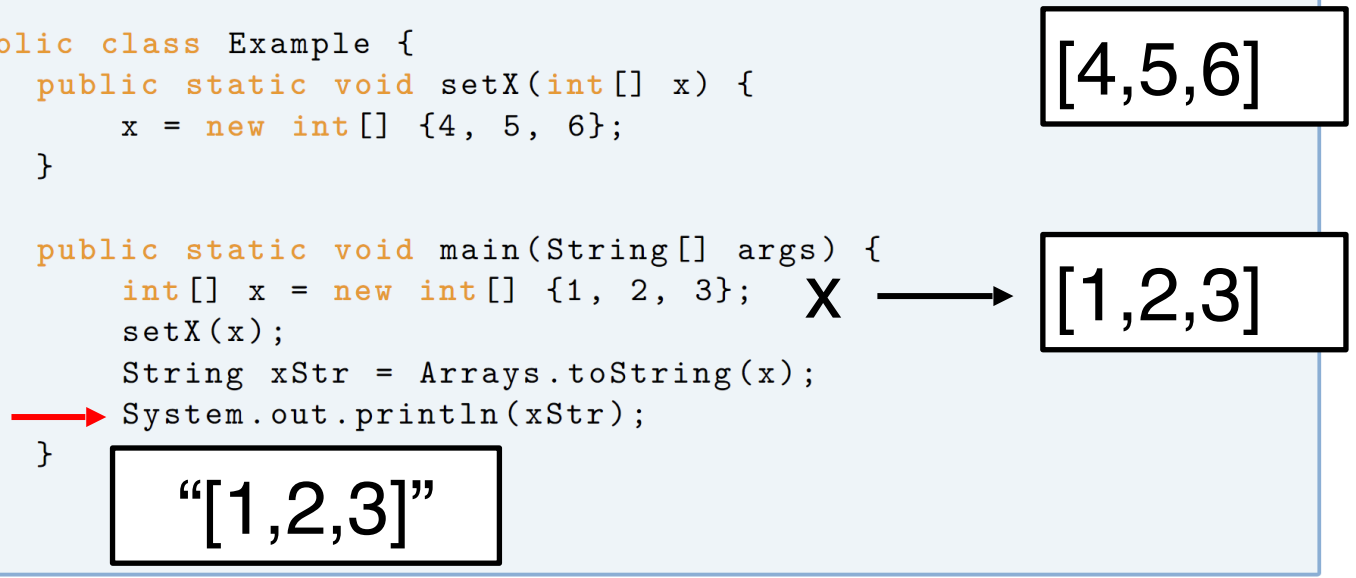
Value vs Reference - Example

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```



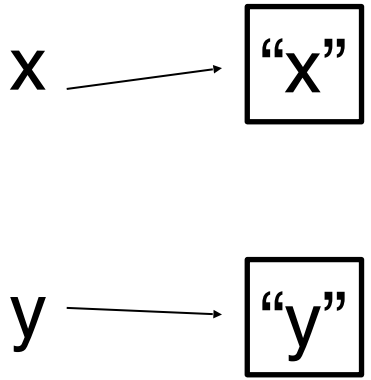
Value vs Reference - Example

```
1  import java.util.Arrays;
2
3  public class Example {
4      public static void setX(int[] x) {
5          x = new int[] {4, 5, 6};
6      }
7
8      public static void main(String[] args) {
9          int[] x = new int[] {1, 2, 3};
10         setX(x);
11         String xStr = Arrays.toString(x);
12         System.out.println(xStr);
13     }
14 }
```



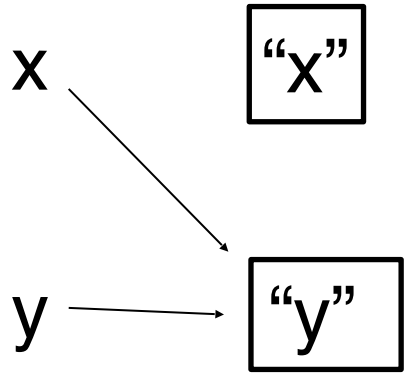
Value vs Reference - Example

```
1  String x = "x";
2  String y = "y";
3
4  //Assignment
5  x = y;
```



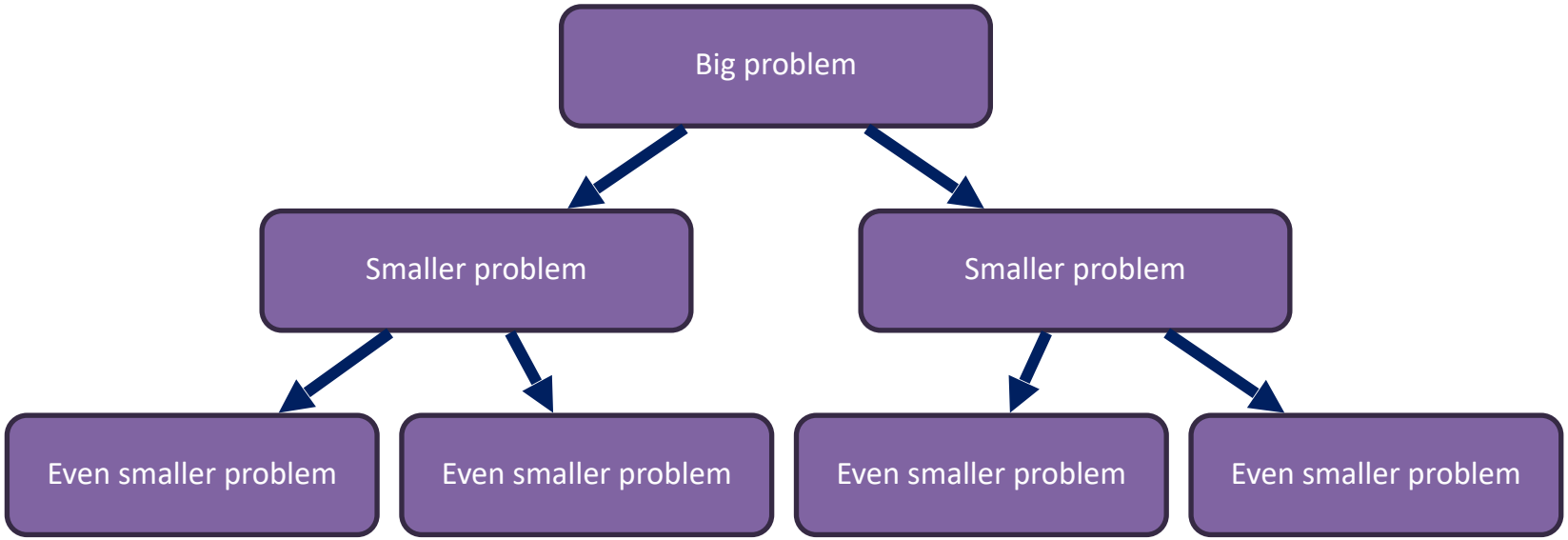
Value vs Reference - Example

```
1  String x = "x";
2  String y = "y";
3
4  //Assignment
5  x = y;
```

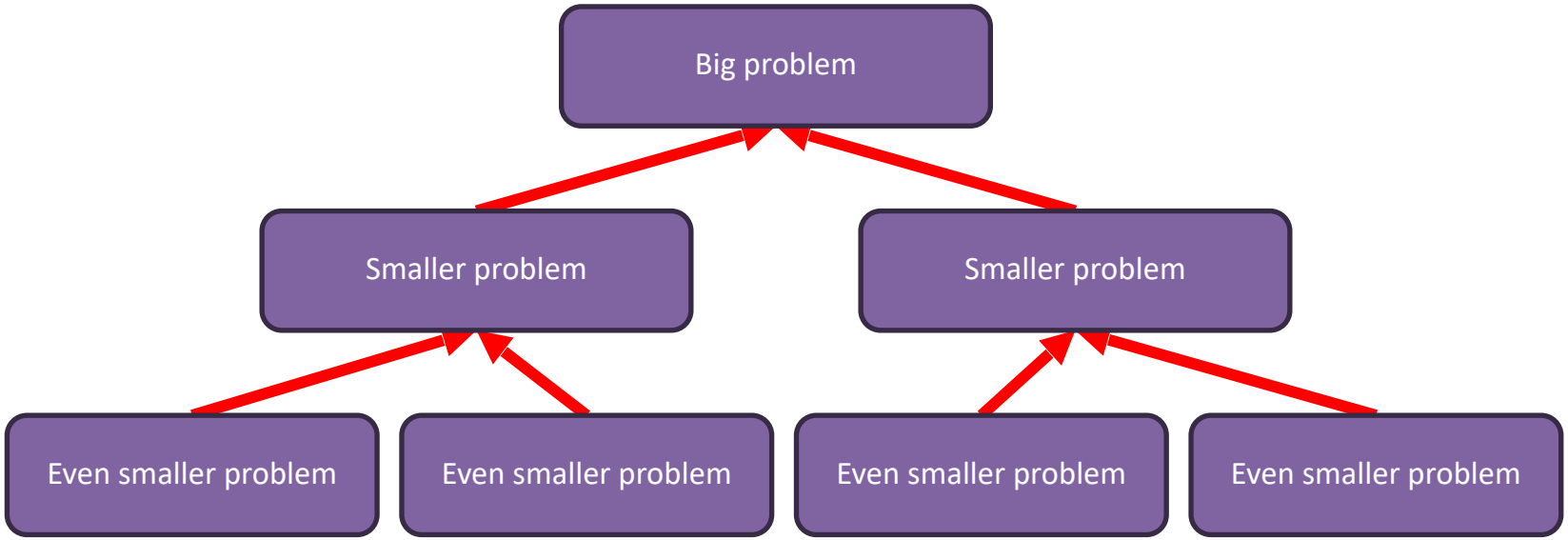


Recursion

Recursion – Basic Principle: Divide and Conquer



Recursion – Basic Principle: Divide and Conquer



Decrease-and-Conquer - Factorial

factorial(3)

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```

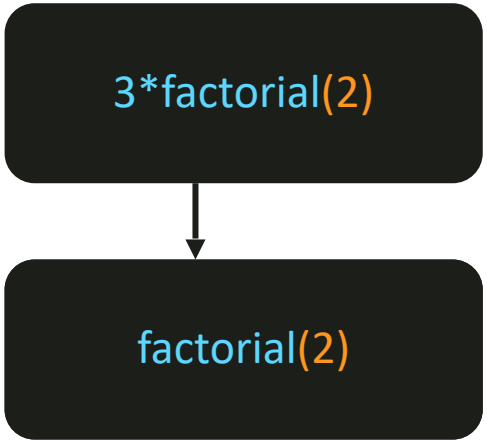
Decrease-and-Conquer - Factorial

3*factorial(2)

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```

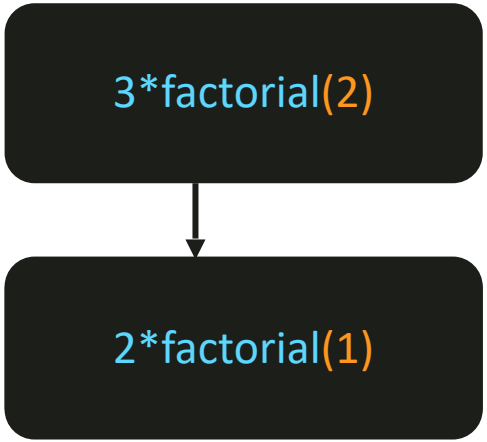
Decrease-and-Conquer - Factorial

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



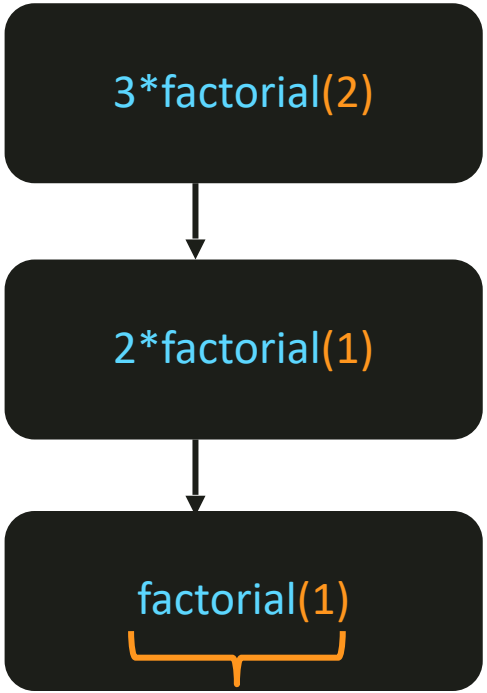
Decrease-and-Conquer - Factorial

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



Decrease-and-Conquer - Factorial

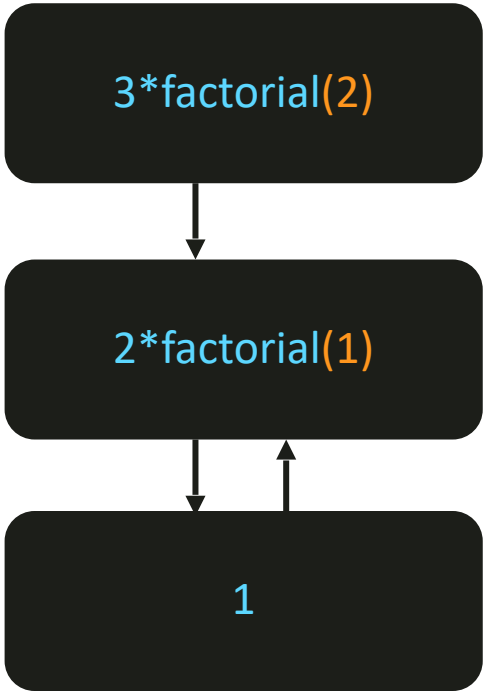
```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



Base Case!

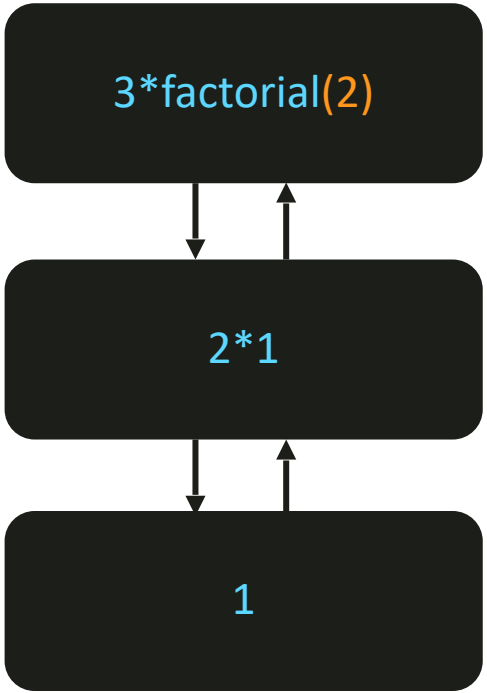
Decrease-and-Conquer - Factorial

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



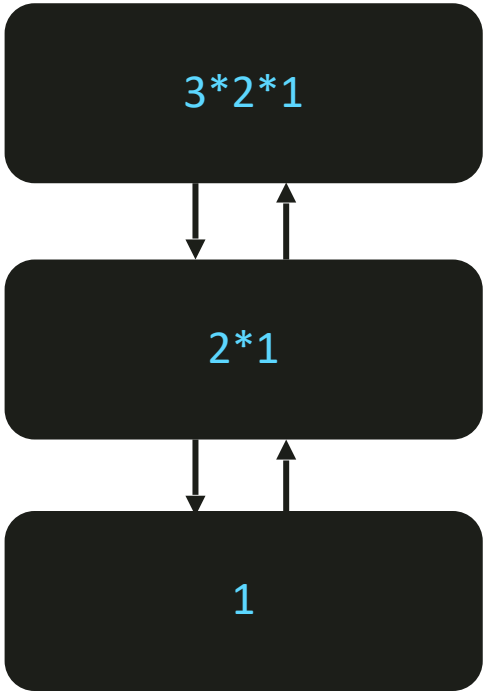
Decrease-and-Conquer - Factorial

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



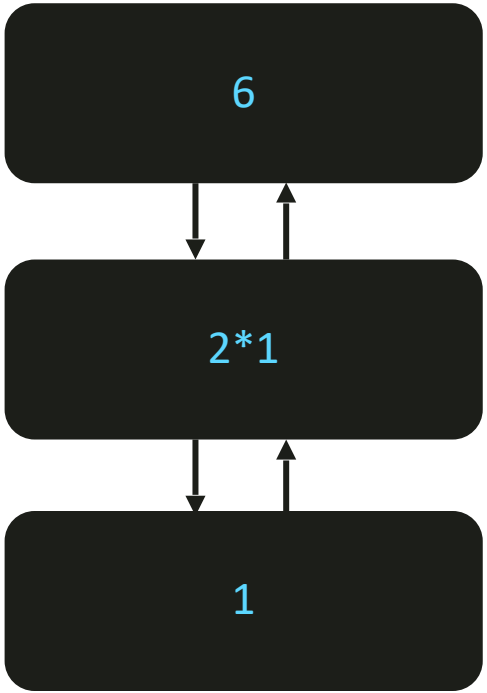
Decrease-and-Conquer - Factorial

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```

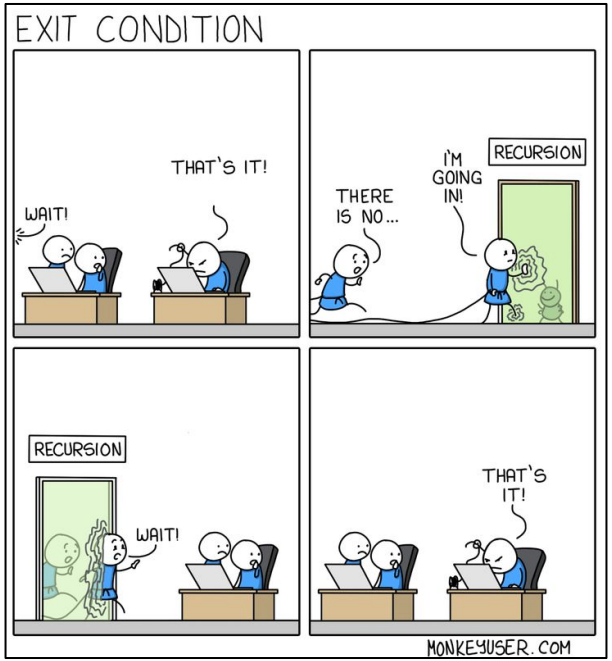


Decrease-and-Conquer - Factorial

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



Recursion in Java – Base Case




```
Exception in thread
"main"
java.lang.StackOverfl
owError
```

Recursion – Step by Step

Problem we want to solve

Exercise: Write a function sum(n) that, given a nonnegative number, sums all nonnegative numbers up to and including n.

Examples:

```
sum(0) = 0
sum(1) = 1 = 1 + 0
sum(4) = 10 = 4 + 3 + 2 + 1 + 0
```

We now want to solve this recursively.

Steps

1. What is the simplest possible input?
2. Try examples and visualize them
3. Derive larger examples from smaller examples
4. Generalize the pattern
5. Write the code

Steps

1. What is the simplest possible input?

   -> Base Case

2. Try examples and visualize them
3. Derive larger examples from smaller examples
4. Generalize the pattern

   -> recursive step

5. Write the code

1. The simplest input

- Here it is n = _ : sum(n) = __
- The simplest input is often our base case later.
- The base case is the only case of a recursive function in which we give a direct solution.
- A function can also have multiple base cases.
- All other solutions build on the base case.

1. The simplest input

- Here it is n = 0: sum(0) = 0
- The simplest input is often our base case later.
- The base case is the only case of a recursive function in which we give a direct solution.
- A function can also have multiple base cases.
- All other solutions build on the base case.

Steps

1. What is the simplest possible input?

   -> Base Case

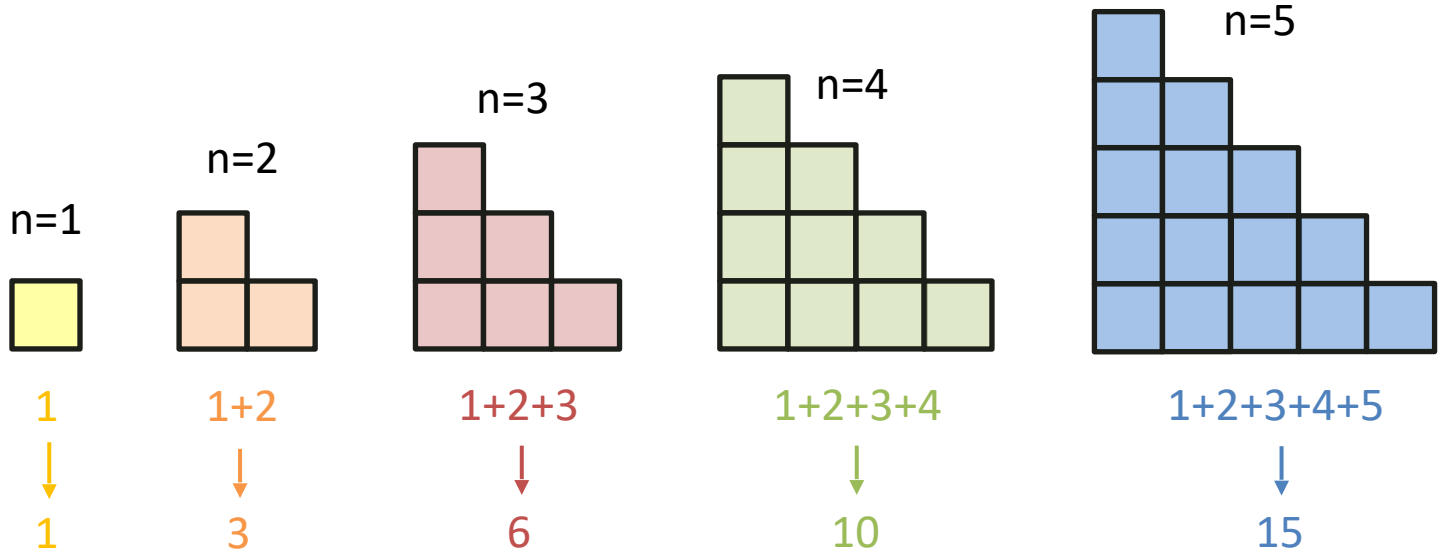
2. Try examples and visualize them
3. Derive larger examples from smaller examples
4. Generalize the pattern

   -> recursive step

5. Write the code

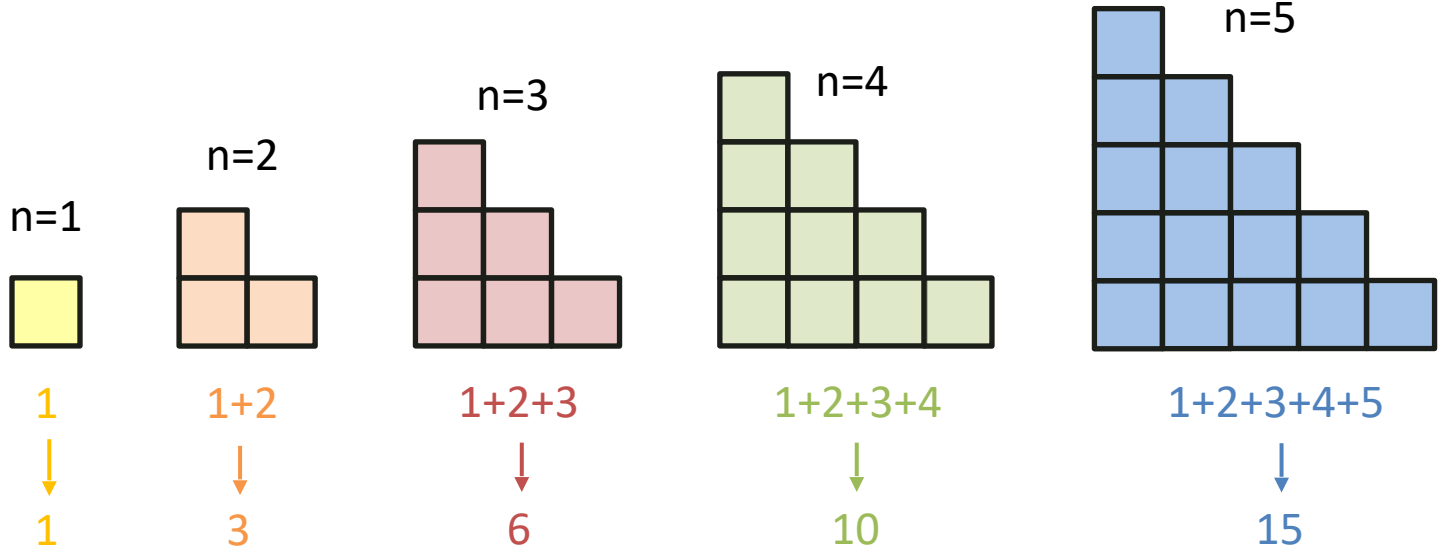
2. The examples

Visualize how the function's inputs and outputs are connected



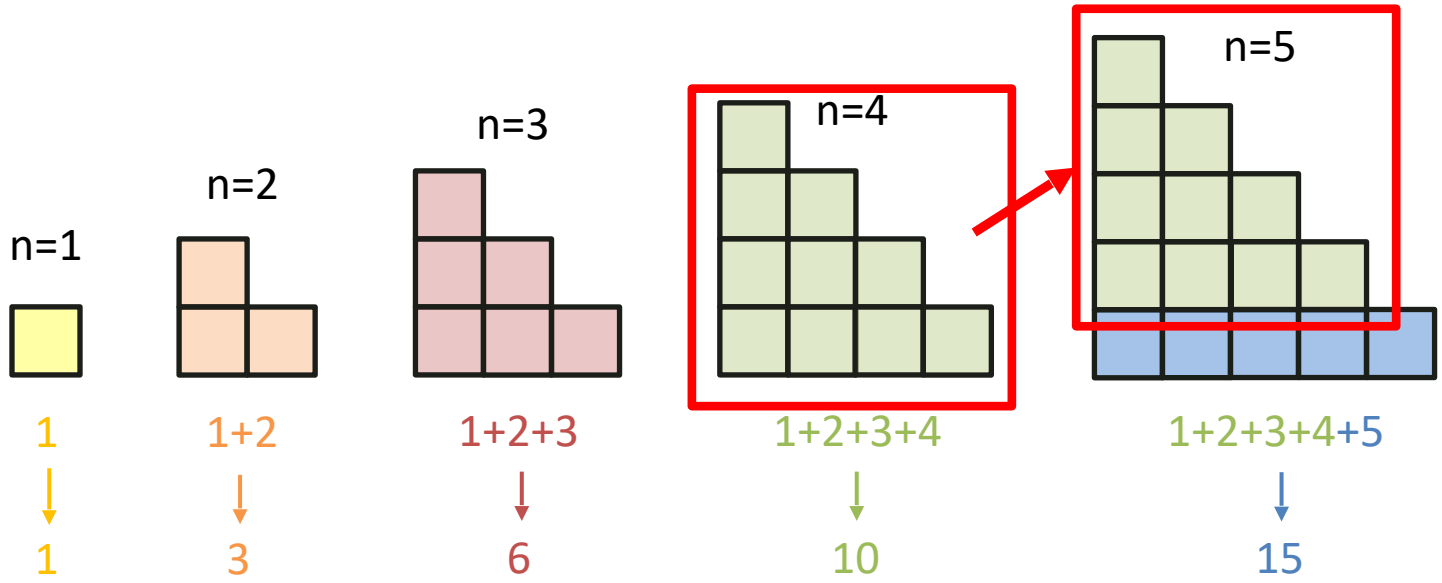
3. Recognize the connection

"If I had been given the answer for n=4, how do I get the answer for n=5? From n=3 to n=4? ..."



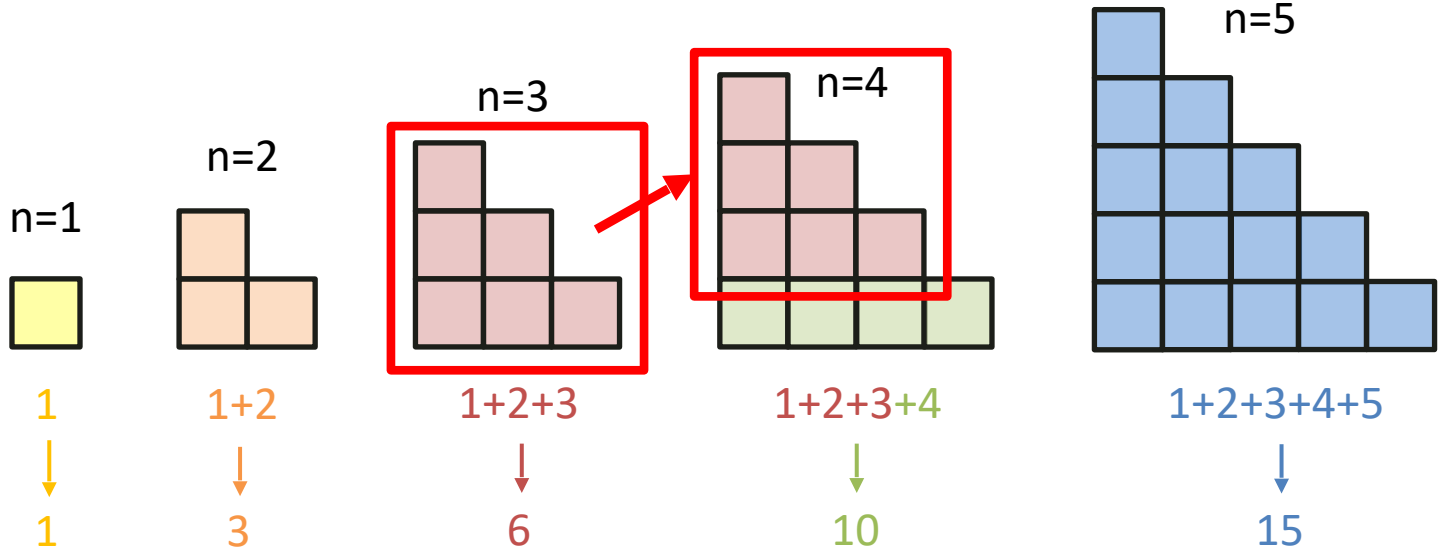
3. Recognize the connection

"If I had been given the answer for n=4, how do I get the answer for n=5? From n=3 to n=4? ..."



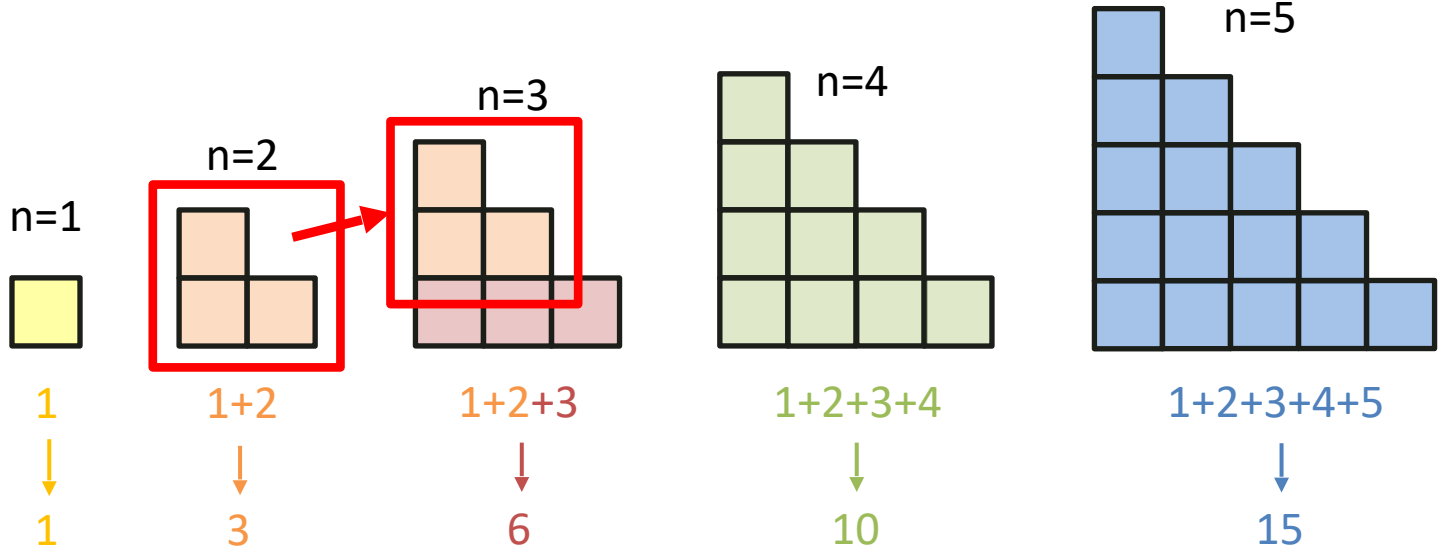
3. Recognize the connection

"If I had been given the answer for n=4, how do I get the answer for n=5? From n=3 to n=4? ..."



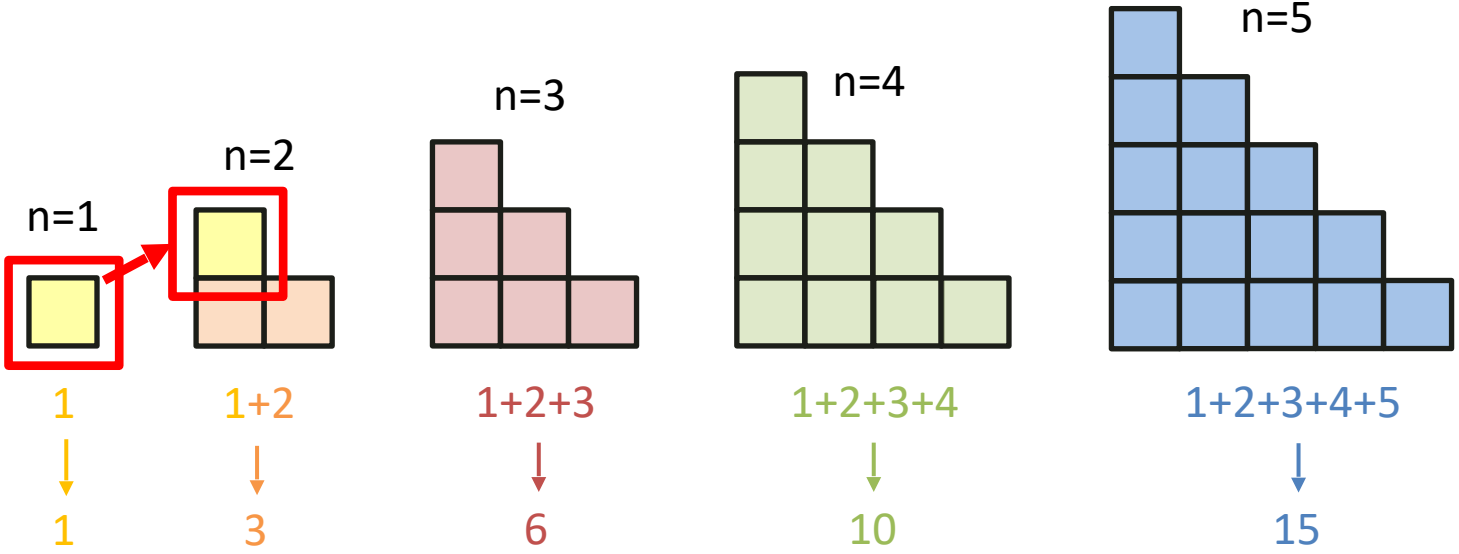
3. Recognize the connection

"If I had been given the answer for n=4, how do I get the answer for n=5? From n=3 to n=4? ..."

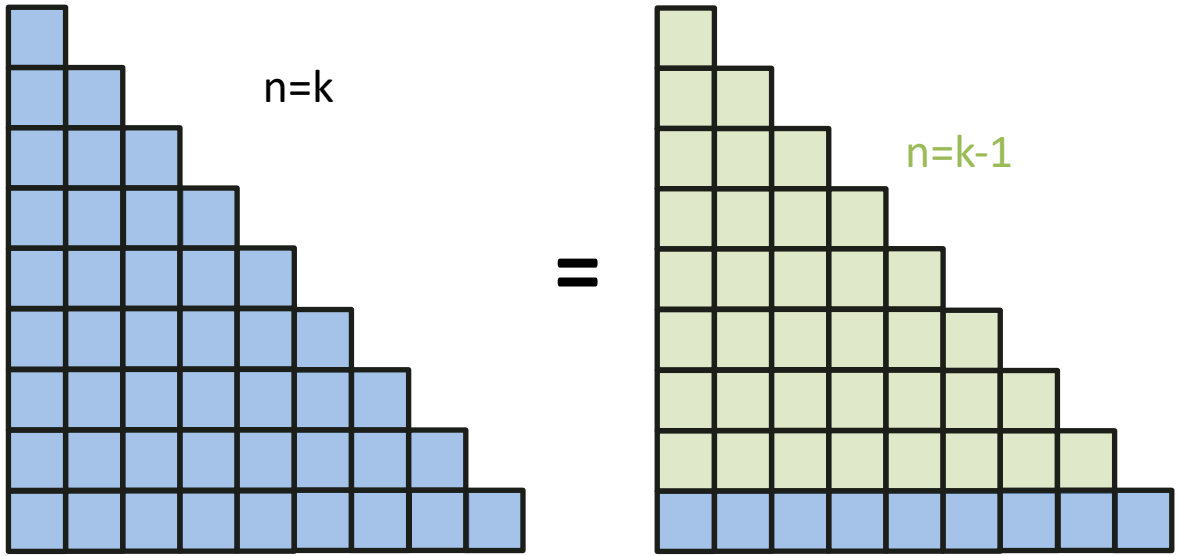


3. Recognize the connection

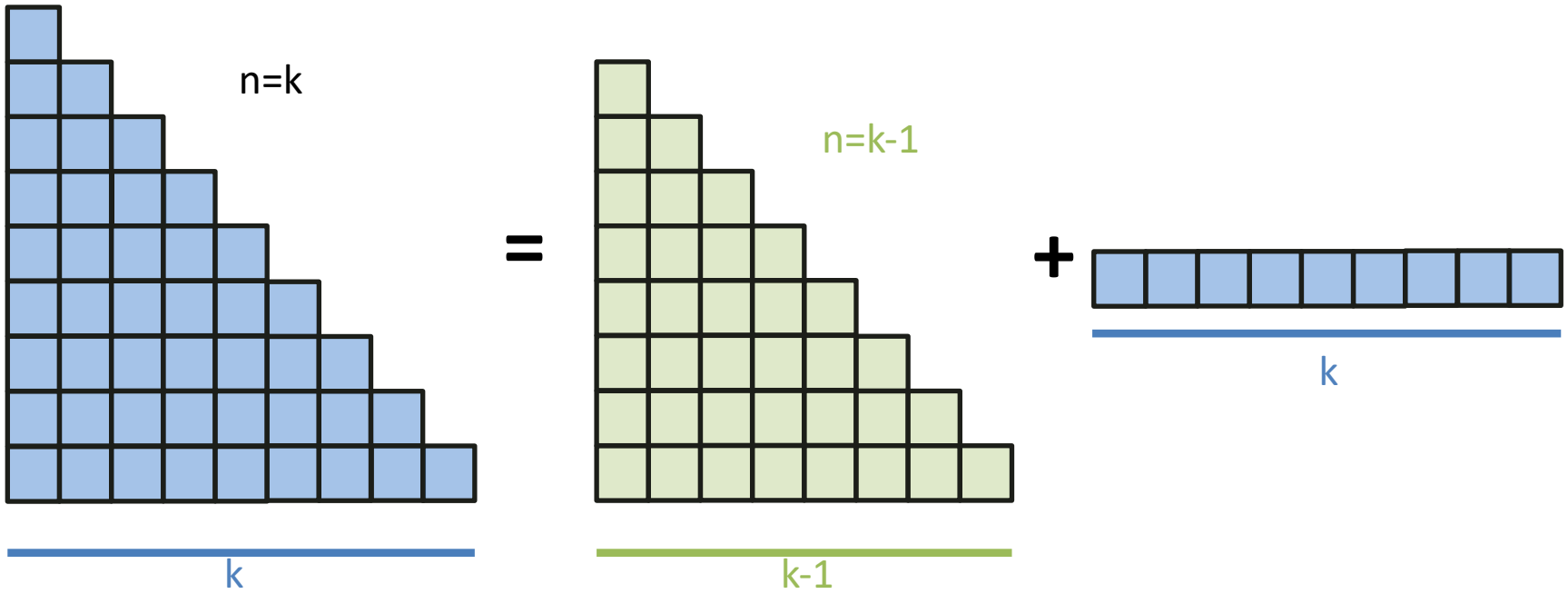
"If I had been given the answer for n=4, how do I get the answer for n=5? From n=3 to n=4? ..."



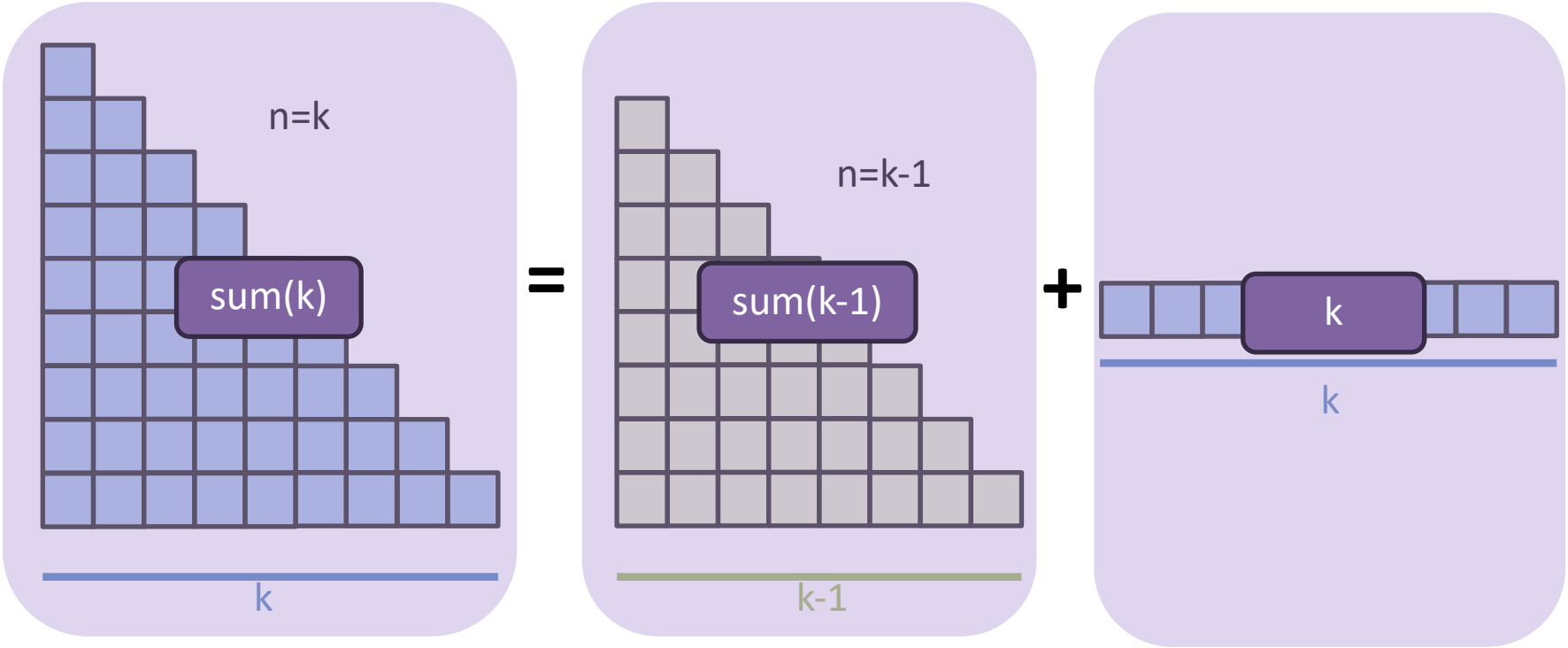
4. The pattern



4. The pattern



4. The pattern



Steps

1. What is the simplest possible input?

   -> Base Case

2. Try examples and visualize them
3. Derive larger examples from smaller examples
4. Generalize the pattern

   -> recursive step

5. Write the code

5. The code

What did we find out?

In the 1st step we found the base case:

If n=0, then sum(n) = 0

After that we found the recursion pattern:

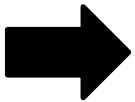
sum(n) = sum(n-1)+n

We can now rewrite this in code

5. Base Case + Pattern → Code

If n=0: sum(n) = 0

Otherwise: sum(n) = sum(n-1)+n



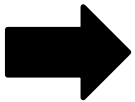
```
public static int sum(int n) {


}
```

5. Base Case + Pattern → Code

If n=0: sum(n) = 0

Otherwise: sum(n) = sum(n-1)+n



```
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }

}
```

5. Base Case + Pattern → Code

If n=0: sum(n) = 0

Otherwise: sum(n) = sum(n-1)+n



```
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }

}
```

5. Base Case + Pattern → Code

If n=0: sum(n) = 0

Otherwise: sum(n) = sum(n-1)+n



```
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }
    return sum(n-1) + n;
}
```

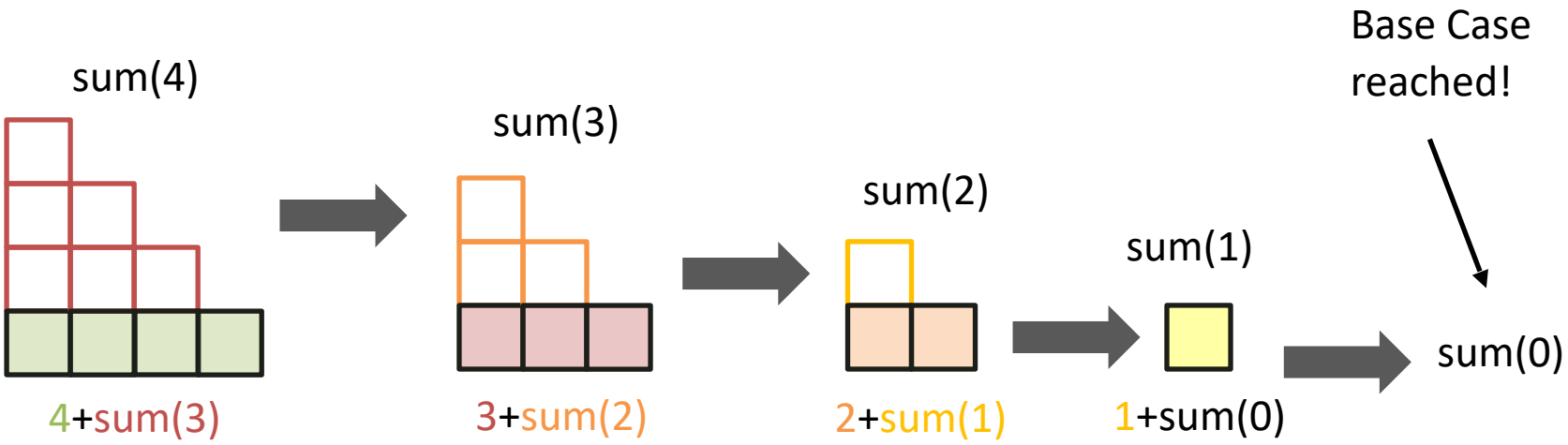
5. Base Case + Pattern -> Code

```
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }
    return sum(n-1) + n;
}
```

Questions?

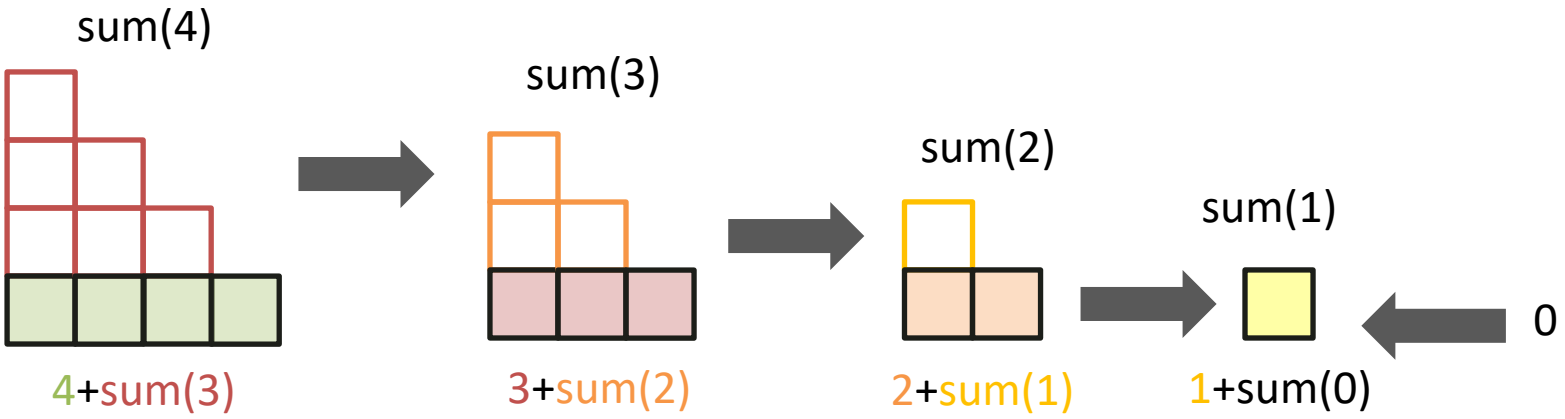
What happens when I run this code?

1. Recursive calls



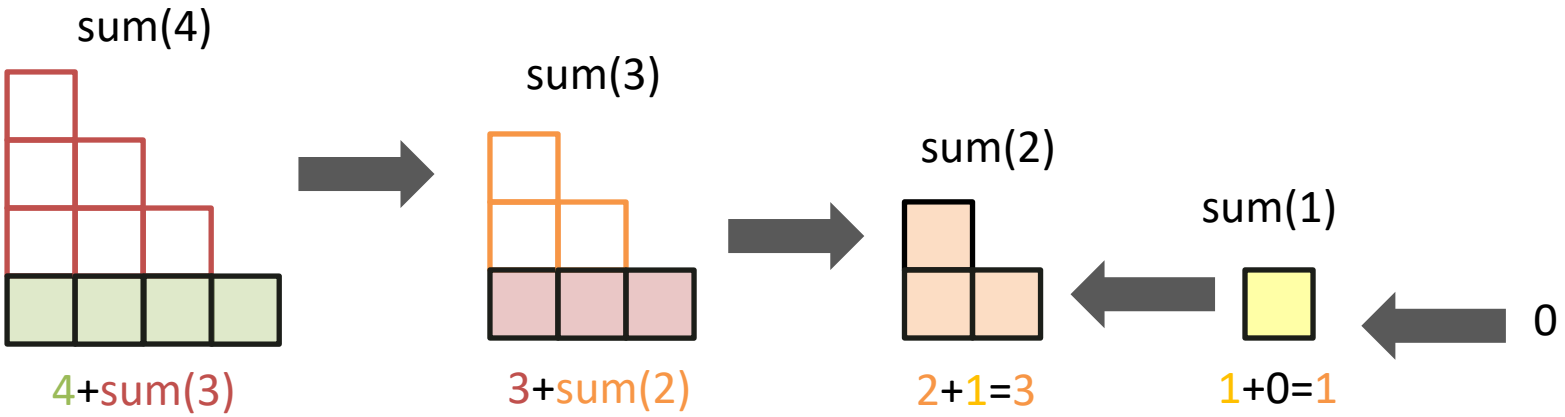
What happens when I run this code?

2. Resolve



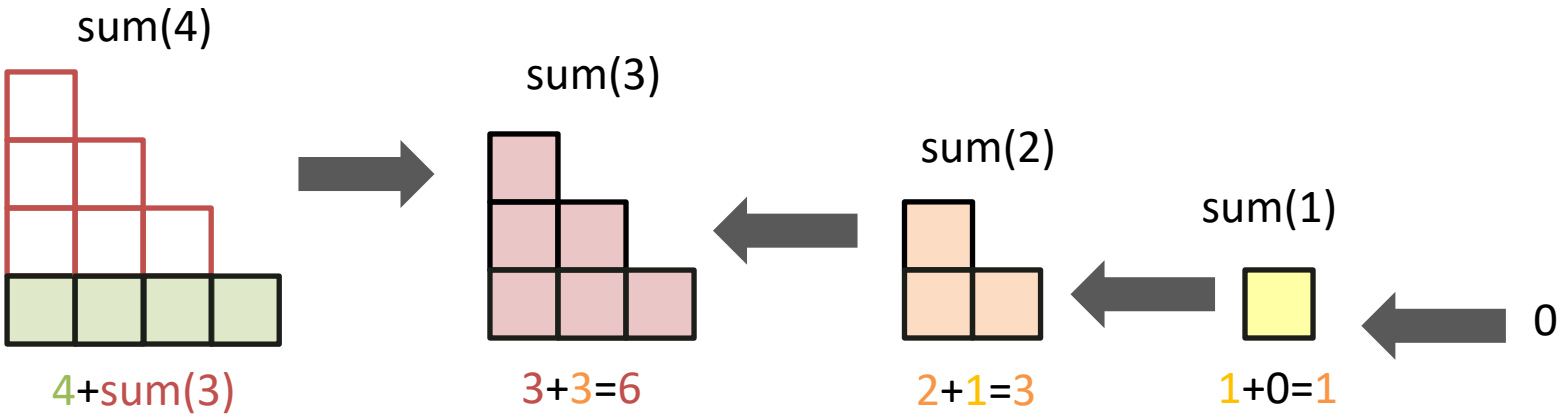
What happens when I run this code?

2. Resolve



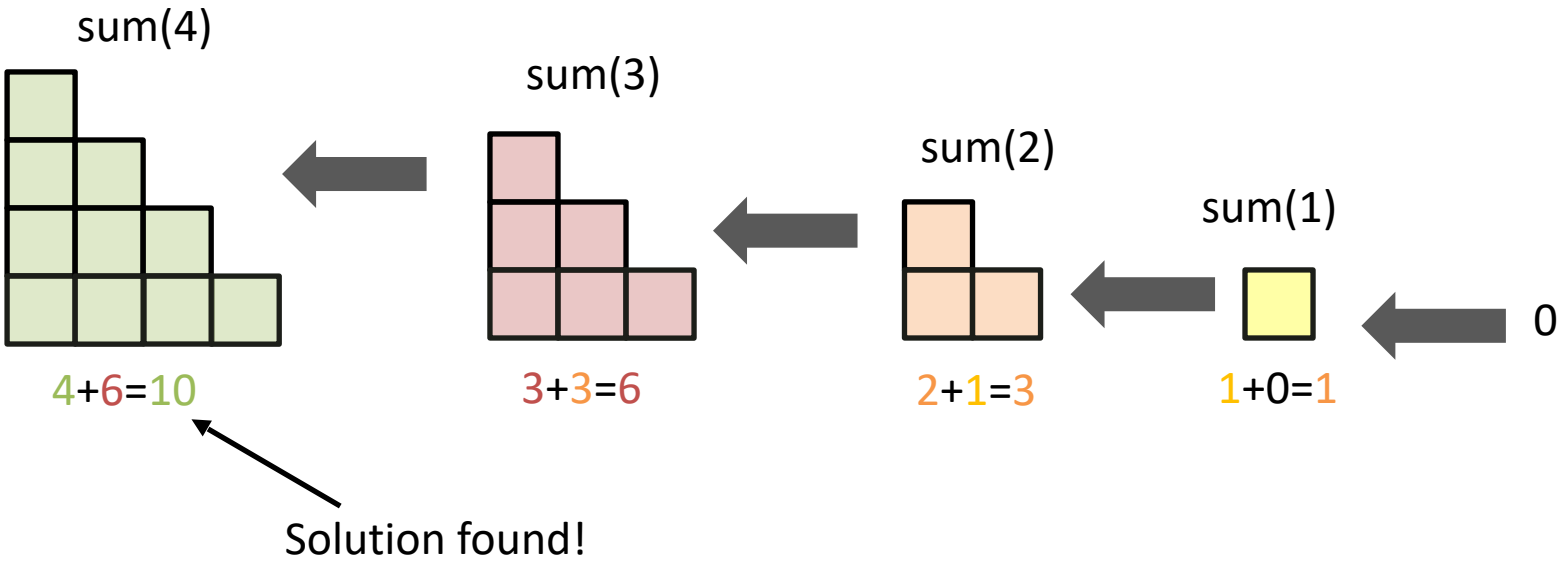
What happens when I run this code?

2. Resolve



What happens when I run this code?

2. Resolve



Recursion in Java – Exercise

Create a class `CheckIsPalindromRecursive`.

Implement a **static, recursive** method `IsPalindrome` that:

- accepts a String
- returns a **boolean** indicating whether the entire String is a palindrome.

Base case of the recursion:

- If the String length is 0 or 1, return `true`.

Recursive case:

- Take the **first** and **last** character of the String.
- Check whether they are **equal**.
- Combine the result with a logical AND with a call to `IsPalindrome` on the substring without the first and last character.

Tip: Use `str.charAt(i)`, `str.substring(i, j)`

What will happen here?

```java
public class checkIsPalindrome {
    public static void main(String[] args) {
        boolean result = isPalindrome("baab");
        System.out.println(result);
    }

    public static boolean isPalindrome(String word) {
        int wordLength = word.length();
        boolean firstAndLast = word.charAt(0) == word.charAt(wordLength - 1);
        String withoutFirstAndLast = word.substring(1, wordLength - 1);
        return isPalindrome(withoutFirstAndLast) && firstAndLast;
    }
}
```

Error? Infinite recursion? Correct execution? Something else?

What will happen here?

```java
public class checkIsPalindrome {
    public static void main(String[] args) {
        boolean result = isPalindrome("wow");
        System.out.println(result);
    }

    public static boolean isPalindrome(String word) {
        int wordLength = word.length();
        boolean firstAndLast = false;
        String withoutFirstAndLast = "";
        if (wordLength != 1 && wordLength != 0) {
            withoutFirstAndLast = word.substring(1, wordLength - 1);
            firstAndLast = word.charAt(0) == word.charAt(wordLength - 1);
        }
        return isPalindrome(withoutFirstAndLast) && firstAndLast;
    }
}
```

What will happen here?

```java
public class checkIsPalindrome {
    public static void main(String[] args) {
        boolean result = isPalindrome("wow");
        System.out.println(result);
    }

    public static boolean isPalindrome(String word) {
        int wordLength = word.length();
        if (wordLength < 1) {
            return true;
        }
        boolean firstAndLast = false;
        String withoutFirstAndLast = "";
        if (wordLength != 1 && wordLength != 0) {
            withoutFirstAndLast = word.substring(1, wordLength - 1);
            firstAndLast = word.charAt(0) == word.charAt(wordLength - 1);
        }
        return isPalindrome(withoutFirstAndLast) && firstAndLast;
    }
}
```

What will happen here?

```java
public class Palindrome {
    public static boolean isPalindrome(String word) {
        if (word.length() <= 1)
            return true;
        boolean firstAndLast = word.charAt(0) == word.charAt(word.length() - 1);
        String withoutFirstAndLast = word.substring(1, word.length() - 1);
        return firstAndLast && isPalindrome(withoutFirstAndLast);
    }
}
```

Logical Reasoning I

Stronger and weaker assertions

- We can distinguish stronger and weaker assertions
  - If P1 $\Longrightarrow$ P2 holds, then P1 is stronger than P2 (and P2 is weaker than P1)

- The strongest assertion is false, because false implies everything
- The weakest assertion is true, because true only implies true

Hoare Triple Syntax

- Consists of two Assertions and one Programm

```
{P} S {Q}
```

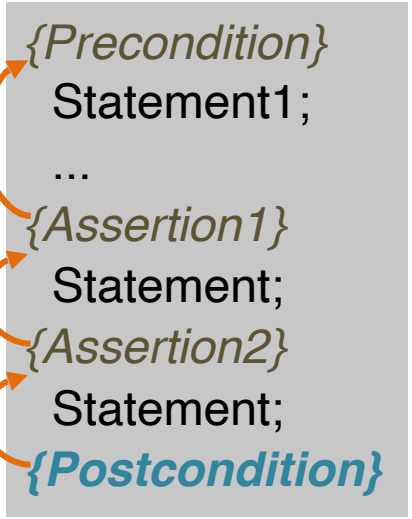
- P is the precondition
- S is the statement
- Q ist the postcondition

Two typical Exercises

- **Is this Hoare Triple valid?**
  - Does the code with the precondition imply the postcondition

- **What is the Weakest Precondition?**
  - Postcondition is given
  - Code is given

Backward reasoning: approach



- Start: choose (know, guess) a meaningful postcondition
- Step by step: derive the previous assertion by including the effect of the previous statement
- Goal: derive a **necessary** and **sufficient** precondition

Backward = "Which precondition does my code need so that it can give the chosen guarantees (postcondition)?"

Weakest Precondition

- The weakest precondition is the weakest precondition that implies the postcondition.
  - If the postcondition is { true }, then { true } is the weakest precondition. Everything implies the postcondition, including in particular the weakest condition true.
  - If the postcondition is { false }, then { false } is the weakest precondition. Only { false } implies the postcondition, so it is the weakest (and only) precondition.
- The rules introduced for backward reasoning directly produce the weakest preconditions.

Weakest Precondition – Simple Example

{ }

$x = y*y$ _____

{x>4}

Weakest Precondition – Simple Example

{ }

$x = y*y$ ——— $\{x>4\}$

Weakest Precondition – Simple Example

{ }

$x = y*y$ ——— $\{y*y>4\}$

Weakest Precondition – Simple Example

{y*y>4}

x = y*y ———————

Weakest Precondition – Simple Example

{|y|>2}

x = y*y ———————

Weakest Precondition – Second Example

{ }

y = x+3 ———————

z = y+1 ———————

{z>4}

Weakest Precondition – Second Example

{ }

y = x+3

z = y+1 {z>4}

Weakest Precondition – Second Example

{ }

y = x+3

z = y+1 {y+1 >4}

Weakest Precondition – Second Example

{ }

y = x+3 ——— {y+1 >4}

z = y+1 ———

Weakest Precondition – Second Example

{ }

y = x+3 ——— {x+3+1 >4}

z = y+1 ———

Weakest Precondition – Second Example

{ }

y = x+3 ——— {x >0}

z = y+1 ———

Weakest Precondition – Second Example

$\{x > 0\}$

$y = x+3$ ———

$z = y+1$ ———

Weakest Precondition - Example

{ }

x = a - 4;

{x>0}

Weakest Precondition - Example

{ }

x = y + z;

y = y - 5;

{y < 0}

Weakest Precondition– Exam Example

```
{                    }

  w = a

  x = w + b;

  y = x * 2;

{y > 0 && b > 10}
```

Weakest Precondition– Exam Example

{ }

p = 3 * q

p = p + 1;

{p > 15}

Kahoot!

Exercise 2: Even and Odd Digits

```
boolean evenEqualsOdd(int n)
```

Return whether the decimal representation of `n`, where $n \geq 0$, contains the same number of even and odd digits.

Examples: `1234` → `true` (even: 2, 4; odd: 1, 3), `10` → `true`, `2468` → `false`, `7` → `false`, and `0` → `false`. The number 0 has one digit, and this digit is even.

Hint: Write a recursive helper method `int evenMinusOdd(int n)` that returns the number of even digits minus the number of odd digits. The last digit of `n` is `n % 10`, and `n / 10` removes it. Check the base case carefully with `n = 0`.

Review

Exercise 1: Sieve of Eratosthenes

Write a program "Sieb.java" that reads in a number *limit* and outputs the number of prime numbers that are greater than 1 and less than or equal to *limit*. To do this, first determine all prime numbers that are less than or equal to *limit*. You can solve this subproblem with the Sieve of Eratosthenes. The Sieve of Eratosthenes finds prime numbers up to n. One considers all numbers from 2 to n and first crosses out all multiples of the first number (2). Then one moves on to the next number that has not been crossed out (3) and repeats crossing out its multiples. This is done until it has been done for all numbers. The prime numbers remain. You can use a `boolean` array to store which numbers are prime and which are not. After that, you can determine the number of prime numbers found using this array.

Example: For $limit = 13$, your program should output 6 (prime numbers: 2, 3, 5, 7, 11, 13).
Hint: When crossing out multiples, it is sufficient to consider possible divisors from 2 to $\sqrt{n}$, since every composite number $\leq n$ has at least one divisor $\leq \sqrt{n}$.

Exercise 2: Arrays

1. Implement the method `ArrayUtil.zeroInsert(int[] x)` in the file "ArrayUtil.java". The method takes an array `x` as an argument and returns an array. The returned array should have the same values as `x`, except: if a positive number directly follows a negative number or if a negative number directly follows a positive number, then a 0 is inserted between them.

   Examples:

   - If `x` is equal to `[3, 4, 5]`, then `[3, 4, 5]` is returned.
   - If `x` is equal to `[3, 0, -5]`, then `[3, 0, -5]` is returned.
   - If `x` is equal to `[-3, 4, 6, 9, -8]`, then `[-3, 0, 4, 6, 9, 0, -8]` is returned.

   Try to implement the method recursively.

Exercise 2: Arrays

2. Implement the method `ArrayUtil.tenFollows(int[] x, int index)`. The method returns a boolean. The method should return `true` if, in the array `x` starting from index `index`, the tenfold value of a number `n` directly follows the number `n`. This only needs to hold for one occurrence of the number `n` starting from index `index` in the array `x` (not for every occurrence). Otherwise, the method should return `false`.

   Examples:

   - `tenFollows([1, 2, 20], 0)` returns `true`.
   - `tenFollows([1, 2, 7, 20], 0)` returns `false`.
   - `tenFollows([3, 30], 0)` returns `true`.
   - `tenFollows([3], 0)` returns `false`.
   - `tenFollows([1, 2, 20, 5], 1)` returns `true`.
   - `tenFollows([1, 2, 20, 5], 2)` returns `false`.

The `main` method in `ArrayUtil` prints the example calls above as well as the corresponding result of each method. This lets you check whether your implementations return the correct results. In "ArrayUtilTest.java" in the "test" folder of the exercise template, you will additionally find some unit tests for both methods (for a detailed description of automated testing and running such tests, see Task 4). You may change the `main` method and the tests however you like and/or extend them with your own inputs.

Exercise 3: 2D Arrays

We say that a matrix M is *perfect* if it satisfies all of the following conditions: It is an $n \times n$ matrix, its elements are positive integers with $0 < m_{i,j} \leq n^2$, and every element occurs exactly once, i.e. $m_{x,y} = m_{p,q} \Rightarrow (x = p) \wedge (y = q)$. In addition, all row sums and column sums are equal (i.e. $\sum_{k=0}^{n-1} m_{i,k} = \sum_{k=0}^{n-1} m_{j,k}$ for all i, j and $\sum_{k=0}^{n-1} m_{k,i} = \sum_{k=0}^{n-1} m_{k,j}$ for all i, j with $0 \leq i, j < n$).

Complete the method `boolean checkMatrix(int[][] m)` of the class `Matrix` so that this method returns `true` if the input matrix is *perfect*, and `false` otherwise. You can assume that the parameter `m` is not `null`. You must test all other properties yourself. A matrix is only perfect if all stated properties hold.

Test your program extensively, ideally with JUnit. We have already created a JUnit test for you in the class `MatrixTest`.

Exercise 4: Testing with JUnit

Purpose of the program:

- Output the weekday of a date (after 01.01.1900)
  Example:13.10.2017 → Friday
  However, incorrectly outputs *“The 13.10.2017 is a Sunday”*.
- Takes leap years into account (“Leap year”)

How it works:

1. Checks whether the date is OK
2. Counts the days from 1.1.1900 to the entered date
3. Weekday = days % 7

Exercise 5: Matching Numbers

Implement the method `Match.matchNumber(long A, int M)`. For a number `A` and a nonnegative three-digit number `M`, the method should return the position of `M` in `A`. Let `M` be a number with digits $M_2M_1M_0$ (that is, $M = M_0 + 10 \cdot M_1 + 100 \cdot M_2$), where each digit may be 0. In addition, let `A` be a number such that A_i is the i-th digit of `A` (that is, for the absolute value of `A`, $|A| = \sum_i 10^i \cdot A_i$), where `A` has arbitrarily many leading zeros. The position of `M` in `A` is the smallest number j such that $A_j = M_0$ and $A_{j+1} = M_1$ and $A_{j+2} = M_2$. The method should return `-1` if there is no such j.

Examples:

`matchNumber(32857890, 789)` should return `1`.
`matchNumber(37897890, 789)` should return `1`.
`matchNumber(1800765, 7)` should return `2`.
`matchNumber(1800765, 8)` should return `-1` (the three digits of `8` are `008`).
`matchNumber(75, 7)` should return `1` (because `007` is at position `1` of `0075`).

Exercise 5: Matching Numbers

Implement the computation in the method `int matchNumber(long A, int M)`, which is located in the class `Match`. The method declaration is already given. You can assume that $0 \leq \texttt{M} < 1000$ holds.

In the `main` method of the class `Match`, you will find the examples above as small tests that make example calls to the `matchNumber` method and that you can use as a basis for further tests. In the file `MatchTest.java`, we additionally provide the same tests as JUnit tests. You may also change these as you like.

Tip: The method `Integer.toString(int i)` converts an integer into a string. Alternatively, you can also use `i + ""`.